

Bookkeeping chat on GCP — architecture and local setup

2026-09-22 · @Someone

Overview

A B2C bookkeeping app where the user manages transactions through a chat, deployed on GCP as two Cloud Run services over one Postgres database. The screen has a chat on the left and a bookkeeping table on the right.

What the chat does:

- **Add, edit and delete transactions** in natural language ("add a 12 EUR coffee today").
- **Filter the table** ("show USD expenses over 100 from last month"). The chat sets the filter; the table re-queries.
- **Categorize automatically.** Every new expense or income gets a category; user corrections are remembered per merchant.
- **Analyze spending and suggest savings** from aggregates of the user's own data.
- **Read receipts from images.** Line items are extracted, categorized and shown as proposed rows; nothing is saved until the user confirms.

Constraints that shape the design:

- **Multicurrency without balance.** Each row stores amount + currency as entered. No FX conversion, no running balance.
- **B2C.** Individual users, Google sign-in, data isolated per user (`uid`). No organization tenants.
- **Third-party LLM.** Inference runs on an external vision-capable API, so no GPUs or model hosting on GCP.

Key design decisions

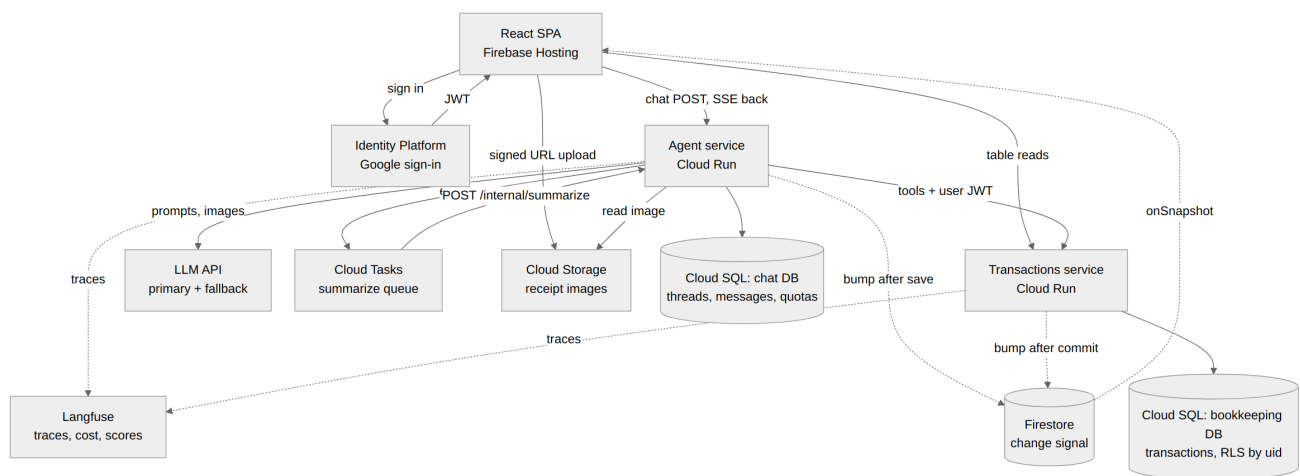
The transactions table is the system of record, and the chat is a controller pointed at it. Most decisions follow from that.

Decision	Choice	Why
Agent framework	LangGraph library inside FastAPI	Receipt ingestion and analysis are real multi-step tool flows; the library gives graph structure, tool orchestration and streaming in-process.
LangGraph Server / Platform	Not used	Its value (threads API, Redis-backed resumable streams, server-side interrupts) serves long-running agents. Here each turn is short. It also needs a commercial license in production and should not run scale-to-zero.
Checkpointing and Redis	Not used	Threads persist in an agent-owned chat database (see Chat memory). Runs never pause server-side, so graph checkpoints add nothing a messages table doesn't.
Streaming	Synchronous SSE on the chat request	Turns finish in seconds. No Pub/Sub or Firestore transport needed.
Tenancy	B2C, scoped by Firebase <code>uid</code>	No org-level customers, so no Identity Platform tenants.
Services	Two Cloud Run services; only one touches Postgres	LLM gets least privilege through an API, and chat and CRUD have different runtime profiles.
Receipt approval	Client-side confirm before commit	Vision extraction is error-prone; proposed rows live in UI state, so no server-side interrupt state.
Analysis across currencies	Per-currency aggregates	No common unit without FX. Add a read-only reporting FX rate later if a unified view is needed.

Open decision: whether analysis stays per-currency or gets a display-only FX conversion. Per-currency is the default in this document.

GCP architecture

The browser talks to two Cloud Run services through Firebase Hosting; only the transactions service holds database credentials.



Secret Manager (LLM, Langfuse and DB credentials) and Cloud Logging/Monitoring attach to both services and are left out of the drawing for readability.

The user's JWT travels from sign-in through the agent to the transactions service, which enforces `uid` scoping itself.

Component	GCP service	Role
Frontend	Firebase Hosting	Serves the React SPA; rewrites <code>/api/chat/*</code> and <code>/api/transactions/*</code> to Cloud Run, same-origin
Auth	Identity Platform (Firebase Auth)	Google sign-in; issues the JWT carrying <code>uid</code>
Agent service	Cloud Run	FastAPI + LangGraph library; SSE streaming; LLM calls; owns the chat DB
Transactions service	Cloud Run	Owns the bookkeeping DB; REST API for CRUD, filtered lists, aggregates, categorization memory
Bookkeeping DB	Cloud SQL Postgres, database <code>bookkeeping</code>	Transactions, category corrections, idempotency keys; RLS by <code>uid</code>
Chat DB	Cloud SQL Postgres, database <code>chat</code> (same instance)	Threads, messages, summaries, preferences, usage counters; RLS by <code>uid</code>
Background jobs	Cloud Tasks	<code>summarize</code> queue; rate-limited, deduplicated, retried
Change signal	Firestore (Native mode)	One <code>sync/{uid}</code> doc of version counters; clients listen and re-fetch
Receipt storage	Cloud Storage	Receipt images, uploaded via signed URL
Secrets	Secret Manager	LLM, Langfuse and DB credentials, mounted as env vars
LLM	Third-party API	Tool calling, categorization, vision extraction, summarization; primary + fallback model
LLM observability	Langfuse Cloud (EU)	Traces, cost per user, quality scores, eval datasets
Infra observability	Cloud Logging + Cloud Monitoring	Structured logs, alerts, budgets
Edge (before launch)	External HTTPS LB + Cloud Armor	WAF, IP rate limiting, custom domain

Services

The transactions service owns data and domain rules; the agent service owns language and orchestration. Categorization, analytics and filtering are functions inside the transactions service, not separate services.

Transactions service

The only service with Postgres credentials. It verifies the Firebase JWT on every request and sets the `uid` for row-level security.

Data model:

```
CREATE TABLE transactions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  uid         text NOT NULL,
  occurred_on date NOT NULL,
  type        text NOT NULL CHECK (type IN ('expense','income')),
  amount_minor bigint NOT NULL CHECK (amount_minor > 0), -- integer minor
  units
  currency    char(3) NOT NULL,          -- ISO 4217, stored as entered
  category    text NOT NULL,
  description  text,
  merchant    text,
  receipt_uri  text,                     -- gs:// path, nullable
  batch_id    uuid,                     -- receipt commit batch, nullable
  created_at  timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ON transactions (uid, occurred_on DESC);
CREATE INDEX ON transactions (uid, currency, category);

CREATE TABLE category_corrections (
  uid        text NOT NULL,
  merchant   text NOT NULL,             -- normalized lowercase
  category   text NOT NULL,
  PRIMARY KEY (uid, merchant)
);

CREATE TABLE idempotency_keys (
  uid        text NOT NULL,
  key        text NOT NULL,             -- tool_call_id or receipt
  batch_id
  request_hash text NOT NULL,
  status      int NOT NULL,
```

```

response      jsonb NOT NULL,
created_at    timestampz NOT NULL DEFAULT now(),
PRIMARY KEY (uid, key)
);
-- purge rows older than 24 h with a daily scheduled job

ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;
CREATE POLICY own_rows ON transactions
  USING (uid = current_setting('app.uid'));
-- same policy on category_corrections and idempotency_keys

```

Each request runs `SELECT set_config('app.uid', $1, true)` inside its transaction, with the `uid` from the verified JWT as a bound parameter. The app connects as a non-owner role so RLS applies.

Money. Amounts are integer minor units, scaled by the currency's ISO 4217 exponent: 12.50 EUR is `1250`, 1,200 JPY is `1200`, 1.250 BHD is `1250`. The API accepts and returns decimal strings (`"12.50"`), rejects more decimals than the currency allows, and converts at the boundary using an exponent table in code. Aggregates sum `amount_minor` per currency, so there is no floating-point rounding anywhere.

Idempotency. Every write accepts an `Idempotency-Key` header. The agent sends the LLM's tool-call ID; the UI sends a client-generated batch ID when confirming a receipt. Inside the same database transaction as the write, the service inserts into `idempotency_keys`. If the key already exists with the same request hash, it returns the stored response without writing again; with a different hash, it returns 409. Chat submissions are deduplicated the same way with a client message ID (see Chat memory).

Method	Path	Purpose
GET	<code>/transactions</code>	List with filter: <code>currency</code> , <code>category</code> , <code>type</code> , <code>from</code> , <code>to</code> , <code>min_amount</code> , <code>max_amount</code> ; keyset pagination
POST	<code>/transactions</code>	Create one or many (batch for receipts); categorizes if category is missing; requires <code>Idempotency-Key</code>
PATCH	<code>/transactions/{id}</code>	Edit; a category change also upserts <code>category_corrections</code> ; requires <code>Idempotency-Key</code>
DELETE	<code>/transactions/{id}</code>	Delete; requires <code>Idempotency-Key</code>
GET	<code>/aggregates</code>	Sums of <code>amount_minor</code> grouped by <code>category</code> , <code>month</code> , per currency

Method	Path	Purpose
POST	<code>/categorize</code>	Internal: category for (merchant, description, amount) , corrections table first, LLM second

Agent service

FastAPI with a LangGraph graph. It owns the chat database but has no access to the bookkeeping database; every tool is an HTTP call to the transactions service with the user's JWT forwarded.

Tool	Calls	Notes
<code>add_transaction</code>	POST <code>/transactions</code>	Asks a clarifying question if amount or currency is missing
<code>edit_transaction</code>	PATCH <code>/transactions/{id}</code>	Resolves "that one" from recent turns or a list call
<code>delete_transaction</code>	DELETE <code>/transactions/{id}</code>	Confirms first for bulk deletes
<code>query_transactions</code>	GET <code>/transactions</code> , GET <code>/aggregates</code>	Feeds analysis and savings suggestions
<code>set_filter</code>	none	Returns a filter object to the UI over SSE
<code>extract_receipt</code>	LLM vision + POST <code>/categorize</code>	Returns proposed rows; never writes

The categorization LLM call can live in the transactions service (it holds the corrections) with its own key from Secret Manager, or be delegated back to the agent. Keeping it in the transactions service means rows created from the table UI get categorized too.

Chat memory

Every thread is persisted in a `chat` database owned by the agent service, so chats survive reloads and devices. Each turn sends the model a token-budgeted context built from that store, never the raw full history.

Storage

A separate `chat` database on the same Cloud SQL instance, accessed only by the agent service. This keeps one owner per data set: the transactions service still owns transactions.

```
CREATE TABLE threads (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  uid         text NOT NULL,
  title       text,
  summary     text,                      -- rolling summary of evicted turns
  summarized_through bigint,            -- last message seq folded into
summary
  working_set jsonb NOT NULL DEFAULT '{}', -- recent entity refs
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE messages (
  seq          bigserial PRIMARY KEY,      -- total order within and across
threads
  client_msg_id uuid,                    -- dedupes resubmitted user
messages
  thread_id    uuid NOT NULL REFERENCES threads(id) ON DELETE CASCADE,
  uid          text NOT NULL,
  role         text NOT NULL CHECK (role IN ('user','assistant','tool')),
  content      jsonb NOT NULL,           -- text, tool calls, tool results,
image refs
  compact      jsonb,                   -- short form used once the message
ages
  token_count  int NOT NULL,
  created_at   timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ON messages (thread_id, seq);
CREATE UNIQUE INDEX ON messages (thread_id, client_msg_id) WHERE
client_msg_id IS NOT NULL;

CREATE TABLE user_preferences (
  uid          text PRIMARY KEY,
  default_currency char(3),
  notes        jsonb NOT NULL DEFAULT '{}'
);
-- RLS on all three tables: uid = current_setting('app.uid')
```

The UI loads `GET /api/chat/threads/{id}/messages` on open and renders full history from the database. What the model sees is built separately, as below.

Context-window handling

The prompt is assembled per turn against a token budget, in this priority order:

Priority	Part	Rule
1	System prompt + tool schemas	Fixed; always included
2	User preferences	Default currency and notes; always included
3	Working set	Last referenced transaction IDs with one-line labels; always included
4	Current user message	Always included; images only on the turn they are uploaded
5	Recent messages	Added newest-first until the budget is spent, never splitting a tool call from its result
6	Rolling summary	Covers everything older than the recent window
-	Output reserve	15–20% of the window kept free for the answer

The techniques that make this work:

1. **Budget by tokens, not message count.** Store `token_count` at write time and fill the window newest-first. A turn with a 200-row list costs far more than ten short turns, and a fixed N would ignore that.
2. **Compact tool results as they age.** Full results stay in the database; the context uses `compact` once a message leaves the last few turns. A 200-row listing becomes "listed 200 USD expenses, Jan–Mar, total 4,120 USD, ids stored". Receipt images are never resent; only the extracted line items remain.
3. **Rolling summary, off the hot path.** When messages fall out of the window, a Cloud Tasks task (see Scalability and resilience) folds them into `threads.summary` with a cheap model: previous summary + evicted messages → new summary, then advances `summarized_through`. The user never waits on summarization.
4. **Working set for references.** After each tool call, the agent records touched transaction IDs with labels ("coffee, 12 EUR, 2026-09-20") in `threads.working_set`, capped at about 20. "Change that one to dining" resolves from this even after the original turn is summarized away.
5. **Numbers come from data, not memory.** The summary records intents and decisions ("user is reviewing September dining"), not amounts as facts. For any figure, the agent re-queries the transactions service, so a lossy summary can't produce a wrong total.

6. **Fallback.** If the provider still returns a context-length error, retry once with the recent window halved.

The transactions table remains the long-term memory of what happened, and `category_corrections` plus `user_preferences` hold what the app has learned about the user. The chat store only has to carry the conversation.

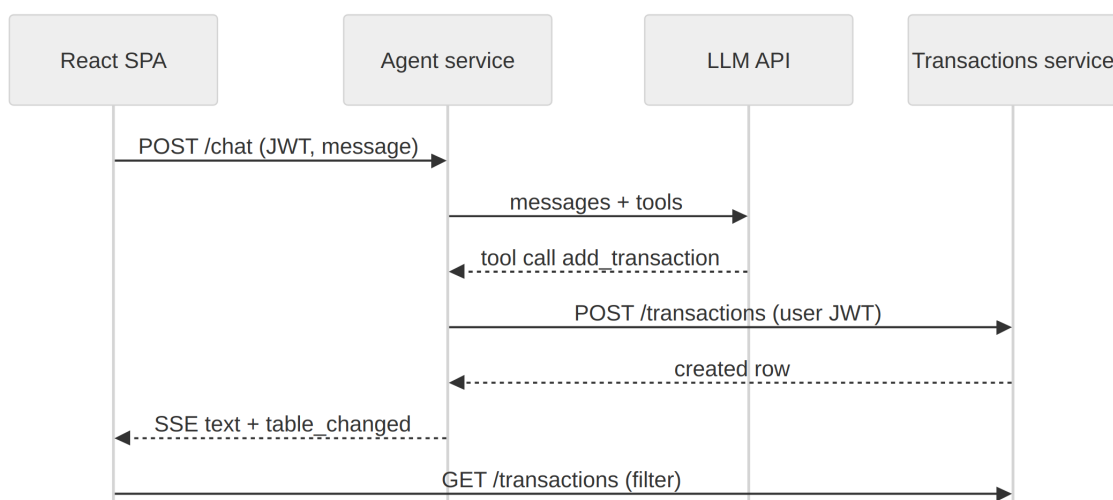
Request flows

Three flows share one spine: the browser posts a chat turn to the agent service and receives tokens and UI events over SSE on the same response.

Every turn starts by saving the user message to the chat DB and assembling context as described in Chat memory. It ends by saving the assistant text, tool calls and tool results, and updating the working set. If older messages fell out of the context window, a summarization task is enqueued to Cloud Tasks.

CRUD and filtering

One tool call per intent. The agent streams text plus a structured event (`table_changed` or `filter_set`), and the UI refreshes the table from the transactions service.



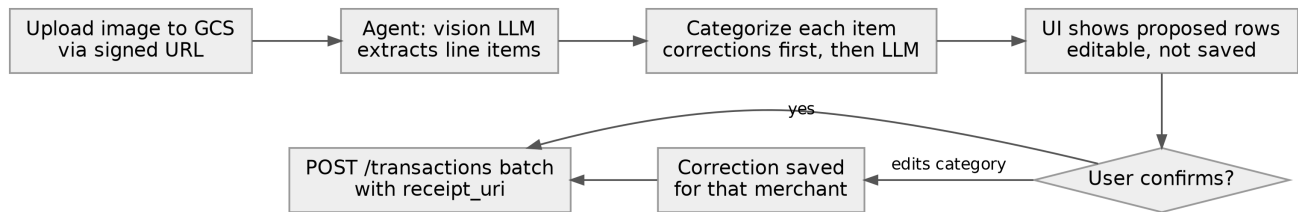
For `set_filter`, the agent calls nothing downstream: the filter object goes to the UI, which re-queries with it.

Analysis and savings suggestions

The agent calls `query_transactions` for per-currency aggregates (by category and month), then the LLM writes observations and options from those numbers. Suggestions stay informational, such as "dining is 31% of EUR expenses, up from 22% last month", not personalized financial advice.

Receipt ingestion

The only multi-step flow, with a human gate before anything is written.



Proposed rows exist only in client state until confirmation, so the approval step needs no server-side interrupt or checkpoint. Rows keep a `receipt_uri` so the image stays linked for audit or re-parsing.

Live updates across tabs and devices

A per-user Firestore document acts as a change signal: services bump a version after each committed write, and every open client listens and re-fetches. Firestore holds only counters, never bookkeeping data or messages.

Same tab vs other clients

- **Same tab:** the agent streams a `table_changed` event with the affected rows over SSE, and the UI applies it immediately. Receipt confirmation updates from its own `POST` response.
- **Other tabs and devices:** they learn about the change from the signal document and re-fetch from the services, which remain the source of truth.

Signal document

Path `sync/{uid}`, one document per user:

```

{
  "transactions_version": 42,
  "thread_versions": { "<threadId>": 17 },
  "threads_version": 5,
  "origin": "<clientId of the tab that caused the write>",
  "updated_at": "<server timestamp>"
}

```

Writer	When	Fields bumped
Transactions service	After any create, edit or delete commits	<code>transactions_version</code>
Agent service	After a turn's messages are saved	<code>thread_versions.<threadId></code>
Agent service	Thread created, renamed or deleted	<code>threads_version</code>

```
# signal.py (both services)
from google.cloud import firestore # honors FIRESTORE_EMULATOR_HOST
db = firestore.Client()

def bump(uid: str, fields: list[str], origin: str | None) -> None:
    try:
        db.document(f"sync/{uid}").set(
            {**{k: firestore.Increment(1) for k in fields},
             "origin": origin, "updated_at": firestore.SERVER_TIMESTAMP},
            merge=True)
    except Exception:
        log.warning("sync bump failed", exc_info=True) # best-effort
```

For a thread version, pass the field path `thread_versions.<threadId>` in an `update()` call (or a nested dict with `set(merge=True)`) so other threads' counters are untouched.

Rules that keep it correct

1. **Bump only after the database commit.** Otherwise a client can re-fetch before the new row is visible and miss it.
2. **Best-effort, not transactional.** A failed bump is logged, not retried into the user's request. Re-fetch on window focus (React Query `refetchOnWindowFocus`) covers the rare miss.
3. **Carry the origin tab.** The UI sends an `X-Client-Id` header (a random ID per tab); services copy it into `origin` . The originating tab skips the re-fetch because it already applied the change, unless the version jumped by more than one, which means another client wrote too.
4. **Re-fetch, don't diff.** A version change triggers a normal `GET` with the current filter. Simple, and correct under concurrent edits from two devices.
5. **Secondary devices get finished chat turns, not token streams.** Tokens stream only to the tab that sent the message.

Client

```
import { getFirestore, doc, onSnapshot, connectFirestoreEmulator } from
"firebase/firestore";

const db = getFirestore(app);
if (import.meta.env.DEV) connectFirestoreEmulator(db, "localhost", 8081);
export const CLIENT_ID = crypto.randomUUID();

export function watchSync(uid: string, onChange: (d: any, prev: any) => void)
{
  let prev: any = null;
  return onSnapshot(doc(db, "sync", uid), snap => {
    const d = snap.data() ?? {};
    if (prev && d.origin !== CLIENT_ID) onChange(d, prev);
    prev = d;
  });
}
// onChange: invalidate the transactions query if transactions_version moved,
// the open thread's messages if its thread_versions entry moved,
// and the thread list if threads_version moved.
```

Security rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /sync/{uid} {
      allow read: if request.auth != null && request.auth.uid == uid;
      allow write: if false;           // services write with the Admin SDK
    }
    match /{document=**} { allow read, write: if false; }
  }
}
```

The signal reveals only that something changed and when; all data still comes from the services with JWT checks and row-level security.

Scalability and resilience

The services are stateless and scale out on Cloud Run; the limits are Postgres connections, the LLM provider's rate limits, and work that runs after a response. Each gets an explicit budget or mechanism below.

Database connection budget

Every Cloud Run instance holds its own pool, so the total can exceed Postgres `max_connections` during a spike. Cap it by construction:

```
(max_instances_tx × pool_tx) + (max_instances_agent × pool_agent) + 20 admin
≤ max_connections
```

Setting	Transactions service	Agent service
<code>--max-instances</code>	20	20
Pool size (<code>max_overflow=0</code>)	5	3 (chat DB only)
<code>--concurrency</code>	80	80
Connections at max	100	60

Set `max_connections` explicitly with a Cloud SQL database flag (for example 200 for the numbers above) rather than relying on the tier default. If the instance runs on Enterprise Plus, enable Cloud SQL managed connection pooling; otherwise the formula is the guard, and PgBouncer is the next step past it. Use `pool_pre_ping=True` and short statement timeouts (for example 5 s for API queries) so a slow query can't hold a connection.

LLM calls

The provider's rate limits are the real ceiling. The agent treats every LLM call as unreliable:

1. **Timeouts:** 15 s to first token, 60 s per call overall.
2. **Retries:** on 429 and 5xx only, at most 2, exponential backoff with jitter, honoring `Retry-After`.
3. **Per-instance concurrency cap:** an async semaphore (for example 40 in-flight LLM calls per instance) so a spike queues inside the instance instead of hammering the provider.
4. **Fallback model:** if retries are exhausted, switch to a secondary model or provider for that turn; if it also fails, stream a clear error. The table keeps working because the transactions service doesn't depend on the agent.
5. **No blind retries after a write:** if a turn fails after a tool call succeeded, the idempotency key (see Services) makes a replayed tool call return the original result instead of writing twice.

Per-user quotas

Quotas live in the chat DB so every instance sees the same count:

```
CREATE TABLE usage_counters (
  uid      text NOT NULL,
  day      date NOT NULL,
  turns    int  NOT NULL DEFAULT 0,
  receipts int  NOT NULL DEFAULT 0,
  tokens   bigint NOT NULL DEFAULT 0,
  PRIMARY KEY (uid, day)
);

-- at the start of a turn: atomic increment-and-read
INSERT INTO usage_counters (uid, day, turns) VALUES ($1, current_date, 1)
ON CONFLICT (uid, day) DO UPDATE SET turns = usage_counters.turns + 1
RETURNING turns;
```

Over the limit, the agent returns 429 with a message the UI shows. Token usage reported by the provider is added after each turn, so heavy receipt users are limited on cost, not just on count.

Background work via Cloud Tasks

With Cloud Run's default request-based billing, CPU is throttled once the response is sent, so work started after the reply can stall. Summarization therefore runs as a Cloud Tasks task, not an in-process background job:

1. After saving a turn, if messages fell out of the context window, the agent enqueues a task on the `summarize` queue targeting `POST /internal/summarize` with `{thread_id, through_seq}`.
2. The task name is derived from `thread_id` and `through_seq`, so duplicate enqueues for the same range are rejected.
3. The handler verifies the task's OIDC token in code, then updates the summary with a compare-and-set on `summarized_through`, so a redelivered task is a no-op.
4. Queue settings cap dispatch rate and concurrency (for example 10/s, 20 concurrent) so summarization never competes with chat turns for LLM quota.

The same queue pattern covers any later async work, such as batch receipt imports or re-categorization.

Other limits

- **Firestore signal:** one `sync/{uid}` bump per request or batch, never per row, keeping well under Firestore's sustained limit of about one write per second per document. A 30-line receipt commit produces one bump.

- **Cold starts:** set `--min-instances=1` on the agent service so the first chat turn after idle doesn't wait for a container start; the transactions service can stay at 0.
- **Next steps past tens of thousands of users:** a read replica for `/aggregates` , and moving the chat DB to its own Cloud SQL instance.

Observability

Langfuse traces every LLM interaction per user and thread, with cost; Cloud Logging and Cloud Monitoring cover the services and infrastructure. The two are joined by one request ID.

Langfuse setup

Decision	Choice
Hosting	Langfuse Cloud, EU region, to start; self-host on GCP later only if data residency requires it (it needs Postgres, ClickHouse, Redis and blob storage)
Projects	One per environment: <code>local</code> , <code>staging</code> , <code>prod</code>
Keys	<code>LANGFUSE_PUBLIC_KEY</code> , <code>LANGFUSE_SECRET_KEY</code> , <code>LANGFUSE_HOST</code> from Secret Manager (<code>.env</code> locally)
SDK	Langfuse Python SDK, pinned; the v3 LangChain integration is <code>langfuse.langchain.CallbackHandler</code>

What is traced

Source	How	Trace attributes
Agent turn (graph, tool calls, LLM calls)	<code>CallbackHandler</code> on the LangGraph run inside an enclosing span	<code>user_id = uid</code> , <code>session_id = thread_id</code> , tags <code>turn</code> , <code>receipt</code>
Receipt extraction	Same turn trace; image sent as a reference, not inlined	<code>receipt_id</code> in metadata
Summarization task	<code>@observe</code> on the task handler	<code>session_id = thread_id</code> , tag <code>summarize</code>

Source	How	Trace attributes
Categorization (transactions service)	@observe around the LLM call	user_id = uid, tag categorize

```
# agent turn
from langfuse import get_client, propagate_attributes
from langfuse.langchain import CallbackHandler

langfuse = get_client()

async def run_turn(uid, thread_id, request_id, state):
    with langfuse.start_as_current_observation(as_type="span", name="chat-
turn"):
        with propagate_attributes(user_id=uid, session_id=thread_id,
                                tags=["turn"], metadata={"request_id":
request_id}):
            handler = CallbackHandler()
            async for event in graph.astream_events(state, config=
{"callbacks": [handler]}):
                yield event
```

The enclosing span with `propagate_attributes` makes user and session IDs land on the top-level trace; setting them only as LangChain metadata can leave them on child spans.

What you get from it

- **Cost per user and per turn**, from token usage and model prices, feeding the quota limits and a daily cost dashboard.
- **Latency breakdown** per turn: time to first token, each tool call, each LLM call.
- **Quality scores:**
 - A category the user later corrects becomes a `categorization_correct = 0` score on that categorization trace; untouched ones score 1 after 7 days. This is a free accuracy metric.
 - Receipt rows edited before confirmation score extraction quality.
 - Optional thumbs up/down in the chat UI as `user_feedback`.
- **Datasets for evals:** failed or corrected traces are added to Langfuse datasets and replayed in CI when prompts or models change.

Privacy in traces

Traces contain financial data, so:

- A masking function strips merchant names and free-text descriptions to hashed or truncated values before export, keeping amounts and categories for debugging.
- Receipt images are never uploaded to Langfuse; only the GCS object path is recorded.
- Retention is set on the project (for example 30 days in prod), and account deletion also deletes that user's traces by `user_id`.

Infrastructure monitoring

- **Structured JSON logs** to Cloud Logging from both services, with `request_id`, `uid`, `thread_id` and the Langfuse trace ID on every line.
- **Alerts** in Cloud Monitoring: 5xx rate above 1%, p95 latency on the transactions API above 1s, Cloud SQL connections above 80% of `max_connections`, Cloud Tasks queue age above 5 minutes, LLM fallback rate above 5%.
- **Budget alerts** on the GCP project and spend caps at the LLM provider.

Security

User isolation is enforced at the transactions service and again in Postgres, never by trusting the agent. Four layers:

1. **User identity propagation.** The agent forwards the user's Firebase JWT on every tool call. The transactions service verifies it with the Firebase Admin SDK and scopes by its `uid`. The agent never asserts "act as uid X" with its own identity, so a bug or injection in the agent cannot reach another user's data.
2. **Service-to-service check at app level.** Firebase Hosting rewrites can only reach Cloud Run services that allow unauthenticated invocation, and the browser calls the transactions service directly for table reads. So IAM `run.invoker` cannot gate these services. Internal-only endpoints (such as `POST /categorize`) instead require a Google-signed ID token from `agent-sa` in `X-Serverless-Authorization`, verified in code.
3. **Row-level security.** Postgres policies on `uid` are the last line if application code misses a filter.
4. **Least privilege for the LLM.** The model acts only through the tool API. A prompt injection hidden in a receipt image can at most trigger valid calls on that same user's data, and receipt rows are never written without user confirmation.

Also required:

- **Per-user quotas on LLM usage.** Every chat turn and receipt costs money. Limit turns and receipts per `uid` per day in the agent service; IP-based limits don't map to one Google account. Counters live in the chat DB so all instances share them (see Scalability and resilience).
- **Upload limits.** Signed URLs are short-lived (for example 5 minutes), restricted to one object path under `receipts/<uid>/`, with content type and size limits (for example images up to 10 MB).
- **Secrets.** LLM keys live in Secret Manager and are mounted as env vars; nothing is baked into images.
- **Frontend.** The browser never holds LLM keys and never talks to Postgres.

Edge and API gateway

No separate API gateway product. Firebase Hosting rewrites cover routing before launch; at launch an external HTTPS load balancer with Cloud Armor takes over routing for the whole app (see Production readiness, item 6).

Gateway function	Where it lives
Single entry point, routing	Firebase Hosting rewrites: <code>/api/chat/**</code> to the agent service, <code>/api/transactions/**</code> to the transactions service
Auth	Each service verifies the Firebase JWT; the transactions service must anyway, as the security boundary
Per-user rate limiting	Agent service quotas keyed on <code>uid</code>
WAF, DDoS, IP rate limiting, custom domain	External HTTPS LB + serverless NEG's + Cloud Armor (before launch)

With the load balancer, raise the backend timeout on the agent route so SSE streams are not cut off. Whichever proxy fronts the agent, confirm it streams SSE without buffering, or tokens arrive in one lump.

The load balancer is the gateway: at launch it is the only public entry point, and both services accept traffic only from it. Other gateway products were considered and rejected:

Option	Why not
GCP API Gateway	Users report that SSE from Cloud Run through it fails with 502s and truncated responses, and that streamed FastAPI responses get buffered (forum , forum). Token streaming is core to the chat. It also can't serve the SPA, and its quotas are per API key, not per user.
Apigee	Supports SSE, but its load balancer timeout defaults to 30 s and must be raised for long streams (docs). Enterprise API management with matching cost, overkill for one B2C frontend.

Private by network, not by IAM. Serverless NEG backends must allow unauthenticated invocation; the ingress setting `internal-and-cloud-load-balancing` is what makes the services unreachable except through the load balancer. That is why the JWT, App Check and ID-token checks in code stay mandatory.

Production readiness

Six items must be in place before public launch; the three checks at the end can only be done on the built system. With both done, the system is launch-ready.

1. Firebase App Check

Until the load balancer restricts ingress, both services are publicly invocable. App Check proves requests come from your real web app.

- **Web provider:** reCAPTCHA Enterprise. The SPA attaches the App Check token as `X-Firebase-AppCheck` on every API call.
- **Verification:** both services verify it with the Firebase Admin SDK, alongside the user JWT. Internal endpoints (Cloud Tasks, `/categorize`) are exempt and use their ID-token check instead.
- **Firestore:** enable App Check enforcement so only your app can open the `sync/{uid}` listener.
- **Rollout:** run in monitor mode for a week, check the verified-request rate, then enforce.
- **Local:** the App Check debug provider with a debug token registered in the `local` project.

2. Delivery pipeline

- **Infrastructure as code:** Terraform for everything in the component table, with state in a versioned GCS bucket. Two GCP projects, `staging` and `prod`, from the same modules.
- **CI/CD:** GitHub Actions authenticating with Workload Identity Federation (no service account keys).
- **Pipeline stages:**
 1. Lint, unit tests, Firestore rules tests (emulator).
 2. Build images, push to Artifact Registry.
 3. Deploy to staging with `--no-traffic`.
 4. Run migrations as a Cloud Run job (Alembic) before shifting traffic.
 5. Smoke tests and the eval gate (item 5) against staging.
 6. Promote to prod: canary 10% of traffic on the new revision, then 100% if error rate and latency hold.
- **Migrations:** expand-then-contract only (add columns, backfill, switch, drop later), so the previous revision always runs against the new schema.
- **Rollback:** shift traffic back to the previous Cloud Run revision; no redeploy needed.

3. Backups and availability

- **Cloud SQL:** automated daily backups retained 30 days, point-in-time recovery with 7 days of logs, deletion protection on the instance.
- **Restore drill:** restore prod backups into staging quarterly and time it.
- **High availability:** regional (HA) Cloud SQL once users pay; zonal before that is an accepted risk.
- **Other stores:** Firestore holds only derived counters and needs no backup. Receipt images follow the retention in item 4.

4. Privacy and data rights

Account deletion runs as a Cloud Tasks job after `DELETE /api/account`, idempotent, each step logged:

1. Bookkeeping DB: transactions, corrections, idempotency keys for the `uid`.
2. Chat DB: threads, messages, preferences, usage counters.
3. Cloud Storage: every object under `receipts/<uid>/`.
4. Firestore: `sync/{uid}`.
5. Langfuse: the user's traces, via its API, by `user_id`.
6. Identity Platform: the user record, last.

Deleted data remains in Cloud SQL backups until they age out (30 days); the privacy policy states this.

Other requirements:

- **Export:** `GET /api/account/export` builds a JSON and CSV archive and returns a short-lived signed URL.
- **LLM provider:** signed DPA, zero-data-retention or no-training setting where offered, EU processing if available.
- **Subprocessors:** the privacy policy lists the LLM provider, Langfuse and Google, and says receipt images are processed by the LLM provider.
- **Retention:** receipt images deleted after a fixed period (for example 24 months) by a bucket lifecycle rule, unless the user deletes earlier.

5. Eval gate

Evals run in CI whenever prompts, model versions or tool schemas change, and nightly. A regression blocks promotion to prod. Results are recorded as Langfuse experiments.

Dataset	Size (start)	Pass threshold
Categorization: labeled transactions, several languages	~300	Accuracy no more than 2 points below the current baseline
Receipt extraction: images with expected line items	~50	Total matches in $\geq 95\%$; line items in $\geq 90\%$
Commands: chat messages with expected tool calls	~150	Correct tool and arguments in $\geq 95\%$
Prompt injection: receipts and messages with hostile instructions	~40	Zero unintended writes or tool calls

Model versions are pinned; upgrading a model is a change that runs the gate.

6. Edge for launch

The external HTTPS load balancer with Cloud Armor is required at launch, not after. Once Cloud Run ingress is set to `internal-and-cloud-load-balancing`, Firebase Hosting rewrites can no longer reach the services, so the load balancer serves the whole app:

- **Routing:** `/api/chat/*` to the agent service NEG, `/api/transactions/*` to the transactions service NEG, everything else to a Cloud Storage backend bucket with Cloud CDN holding the SPA build.

- **Cloud Armor:** preconfigured OWASP rules, per-IP rate limits (stricter on `/api/chat/*`), and geo or bot rules if abuse appears.
- **Timeouts:** backend timeout of 300 s on the agent route for SSE.
- **TLS:** Google-managed certificate on your domain.
- **Sign-in:** keep Google sign-in working on the custom domain by following Firebase's guidance for sign-in on custom domains (auth domain and helper paths).

Internal traffic after the ingress change. Once both services accept only internal and load-balancer traffic, every caller must qualify:

Caller	Target	How it stays allowed
Browser	Both services	Through the load balancer only
Agent service	Transactions service	Direct VPC egress on the agent with all traffic routed through the VPC, so calls to the transactions service count as internal. Without it, these calls are rejected after the ingress change.
Agent and transactions services	Cloud SQL, Firestore, GCS, Langfuse, LLM API	Unaffected: ingress settings restrict who calls Cloud Run, not what Cloud Run calls. Routing all egress through the VPC needs Cloud NAT for the public LLM and Langfuse endpoints.
Cloud Tasks	Agent <code>/internal/summarize</code>	Google treats Cloud Tasks requests as internal; the OIDC token check in code still applies. Confirm in staging.

The staging test for this change: flip ingress, then run the full smoke suite, a chat turn with a tool write, and a summarization task.

Firebase Hosting remains a valid pre-launch and preview setup.

Verify after building

- ☐ Load test (k6 or Locust) at 3× expected peak: connection budget holds, LLM concurrency cap queues instead of failing, quotas enforce.

- ☐ Authorization tests: a second test user attempts to read, edit and delete the first user's transactions, threads and sync doc through every endpoint, and fails.
- ☐ Firestore rules tests with the emulator in CI.
- ☐ Runbooks for every alert in Observability, and a named person or rotation responding to them.

Local development with Docker Compose

The whole system runs locally with one `docker compose up`; only the LLM API stays external. Every GCP piece has a container or emulator stand-in.

Production	Local stand-in
Firebase Hosting rewrites	Caddy on <code>localhost:8080</code> with the same path routing
Identity Platform (Google sign-in)	Firebase Auth Emulator, project <code>demo-bookkeeping</code>
Firestore change signal	Firestore Emulator on port 8081, same security rules
Cloud SQL Postgres	<code>postgres:16</code> with the same schema and RLS
Cloud Storage	<code>fsouza/fake-gcs-server</code>
Cloud Tasks	<code>TASKS_MODE=local</code> : the same handler runs in-process via <code>asyncio.create_task</code> (no CPU throttling locally)
Langfuse	Langfuse Cloud <code>local</code> project (keys in <code>.env</code>); self-hosted Langfuse only if you need offline dev
Secret Manager	<code>.env</code> file read as env vars
Service-to-service ID-token check	Skipped via <code>SKIP_SERVICE_AUTH=true</code>
LLM API	The real API, key from <code>.env</code>

Repository layout

```

bookkeeping/
├─ docker-compose.yml
├─ .env.example
├─ Caddyfile
├─ firebase/
│   └─ Dockerfile
│   └─ firebase.json
├─ db/init/001_schema.sql
├─ gcs/receipts-local/      # empty dir = pre-created bucket
├─ services/
│   └─ transactions/      # FastAPI, Dockerfile
│   └─ agent/             # FastAPI + LangGraph, Dockerfile
└─ web/                   # React + Vite

```

docker-compose.yml

```

services:
  proxy:
    image: caddy:2
    ports: ["8080:8080"]
    volumes: [".:/Caddyfile:/etc/caddy/Caddyfile:ro"]
    depends_on: [web, agent, transactions, fake-gcs]

  web:
    build: ./web
    command: npm run dev -- --host 0.0.0.0 --port 5173
    volumes: [".:/web:/app", "/app/node_modules"]

  firebase:
    build: ./firebase
    ports: ["9099:9099", "8081:8081", "4000:4000"] # auth, firestore,
    emulator UI

  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: bookkeeping
      POSTGRES_USER: owner
      POSTGRES_PASSWORD: owner_pw
    volumes:
      - ./db/init:/docker-entrypoint-initdb.d:ro # creates bookkeeping +
    chat DBs
      - pgdata:/var/lib/postgresql/data
    ports: ["5432:5432"]

```

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U owner -d bookkeeping"]
  interval: 3s
  retries: 20
```

```
fake-gcs:
  image: fsouza/fake-gcs-server
  command: ["-scheme", "http", "-port", "4443", "-backend", "memory"]
  volumes: [".:/gcs:/data"]
```

```
transactions:
  build: ./services/transactions
  env_file: .env
  environment:
    DATABASE_URL: postgresql://app_user:app_pw@postgres:5432/bookkeeping
    FIREBASE_AUTH_EMULATOR_HOST: firebase:9099
    FIRESTORE_EMULATOR_HOST: firebase:8081
    GOOGLE_CLOUD_PROJECT: demo-bookkeeping
    SKIP_SERVICE_AUTH: "true"
  depends_on:
    postgres: { condition: service_healthy }
    firebase: { condition: service_started }
```

```
agent:
  build: ./services/agent
  env_file: .env
  environment:
    CHAT_DATABASE_URL: postgresql://chat_user:chat_pw@postgres:5432/chat
    TRANSACTIONS_URL: http://transactions:8000
    FIREBASE_AUTH_EMULATOR_HOST: firebase:9099
    FIRESTORE_EMULATOR_HOST: firebase:8081
    GOOGLE_CLOUD_PROJECT: demo-bookkeeping
    STORAGE_MODE: local
    TASKS_MODE: local
    STORAGE_EMULATOR_HOST: http://fake-gcs:4443
    RECEIPTS_BUCKET: receipts-local
    PUBLIC_UPLOAD_BASE: http://localhost:8080/gcs
    SKIP_SERVICE_AUTH: "true"
  depends_on:
    postgres: { condition: service_healthy }
    transactions: { condition: service_started }
    fake-gcs: { condition: service_started }
```

```
volumes:
  pgdata:
```

Caddyfile

Paths are forwarded whole, as Firebase Hosting rewrites do, so services define routes under `/api/chat` and `/api/transactions` in both environments.

```
:8080 {
  handle /api/chat/* {
    reverse_proxy agent:8000 {
      flush_interval -1          # stream SSE immediately
    }
  }
  handle /api/transactions/* {
    reverse_proxy transactions:8000
  }
  handle_path /gcs/* {
    reverse_proxy fake-gcs:4443 # same-origin uploads, no CORS
  }
  handle {
    reverse_proxy web:5173      # Vite dev server + HMR
  }
}
```

Firebase Auth Emulator

firebase/Dockerfile:

```
FROM node:20-slim
RUN apt-get update && apt-get install -y --no-install-recommends openjdk-17-jre-headless \
    && rm -rf /var/lib/apt/lists/* && npm install -g firebase-tools
WORKDIR /emulator
COPY firebase.json firestore.rules ./
CMD ["firebase", "emulators:start", "--only", "auth,firestore", "--project", "demo-bookkeeping"]
```

firebase/firebase.json:

```
{
  "firestore": { "rules": "firestore.rules" },
  "emulators": {
    "auth": { "host": "0.0.0.0", "port": 9099 },
    "firestore": { "host": "0.0.0.0", "port": 8081 },
    "ui": { "enabled": true, "host": "0.0.0.0", "port": 4000 }
  }
}
```

A `demo-` project ID runs fully offline with no Firebase login. The Firestore emulator needs the Java runtime; `firebase/firestore.rules` holds the same rules as production (Live updates section), so rule behavior is tested locally.

In the React app, point auth at the emulator in dev only:

```
import { initializeApp } from "firebase/app";
import { getAuth, connectAuthEmulator, GoogleAuthProvider, signInWithPopup }
  from "firebase/auth";

const app = initializeApp({ apiKey: "demo-key", projectId: "demo-
bookkeeping", authDomain: "localhost" });
export const auth = getAuth(app);
if (import.meta.env.DEV) connectAuthEmulator(auth, "http://localhost:9099");

export const signIn = () => signInWithPopup(auth, new GoogleAuthProvider());
```

With `FIREBASE_AUTH_EMULATOR_HOST` set, the Admin SDK in both services accepts emulator tokens, so `verify_id_token` and JWT forwarding work unchanged.

Database init

`db/init/001_schema.sql` creates the tables and policies from the Services section, then a non-owner app role so RLS applies:

```
CREATE ROLE app_user LOGIN PASSWORD 'app_pw';
GRANT SELECT, INSERT, UPDATE, DELETE ON transactions, category_corrections TO
app_user;
```

`db/init/002_chat.sql` creates the agent-owned chat database with the tables from Chat memory and its own role. The two roles cannot read each other's databases.

```
CREATE DATABASE chat;
CREATE ROLE chat_user LOGIN PASSWORD 'chat_pw';
\connect chat
-- threads, messages, user_preferences, usage_counters + RLS policies
GRANT SELECT, INSERT, UPDATE, DELETE ON threads, messages, user_preferences,
usage_counters TO chat_user;
GRANT USAGE ON SEQUENCE messages_seq_seq TO chat_user;
REVOKE CONNECT ON DATABASE chat FROM PUBLIC;
GRANT CONNECT ON DATABASE chat TO chat_user;
```

Apply the same isolation in the bookkeeping database: `REVOKE CONNECT ON DATABASE bookkeeping FROM PUBLIC` and grant it only to `app_user`.

The init scripts run only on an empty volume. After schema changes, run `docker compose down -v` or adopt a migration tool (Alembic, Flyway) that runs the same way locally and in production.

Code seams that switch by environment

Two small interfaces keep local and production code identical everywhere else.

```
# storage.py (agent service)
import os, datetime
from google.cloud import storage # honors STORAGE_EMULATOR_HOST

BUCKET = os.environ["RECEIPTS_BUCKET"]

def upload_target(uid: str, object_id: str) -> dict:
    name = f"receipts/{uid}/{object_id}.jpg"
    if os.getenv("STORAGE_MODE") == "local":
        base = os.environ["PUBLIC_UPLOAD_BASE"]
        return {"method": "POST", "object": name,
                "url": f"{base}/upload/storage/v1/b/{BUCKET}/o?uploadType=media&name={name}"}
    blob = storage.Client().bucket(BUCKET).blob(name)
    url = blob.generate_signed_url(version="v4", method="PUT",
                                   expiration=datetime.timedelta(minutes=5),
                                   content_type="image/jpeg")
    return {"method": "PUT", "object": name, "url": url}
```

The frontend sends the image with whatever method and URL it receives, so it can't tell the environments apart. On Cloud Run, V4 signing without a key file needs the service account to hold `roles/iam.serviceAccountTokenCreator` on itself.

```
# auth.py (both services)
import os
from firebase_admin import auth, initialize_app
initialize_app()

def user_uid(request) -> str:
    token = request.headers["Authorization"].removeprefix("Bearer ")
    return auth.verify_id_token(token)["uid"] # always on, local too

def check_service_caller(request) -> None:
    if os.getenv("SKIP_SERVICE_AUTH") == "true":
        return # local only
    # verify Google-signed ID token from X-Serverless-Authorization
    ...
```

User JWT verification never switches off. It is the check that enforces isolation, so it must run in every environment.

`.env.example`

```
LLM_API_KEY=
LLM_FALLBACK_API_KEY=
LANGFUSE_PUBLIC_KEY=          # Langfuse Cloud, project "local"
LANGFUSE_SECRET_KEY=
LANGFUSE_HOST=https://cloud.langfuse.com
```

Run it

1. `cp .env.example .env` and set `LLM_API_KEY`.
2. Create the empty bucket directory: `mkdir -p gcs/receipts-local`.
3. `docker compose up --build`.
4. Open `http://localhost:8080` and sign in. The emulator shows a fake Google account picker; add any test account.
5. Inspect users and tokens in the emulator UI at `http://localhost:4000`; connect a SQL client to `localhost:5432`.
6. Reset everything with `docker compose down -v`.

If you later split the receipt worker behind Pub/Sub, add the Pub/Sub emulator (`gcloud beta emulators pubsub start` in a container) to the same file.

Local vs production and deployment

Three things behave differently locally; test them in a staging GCP project before launch.

Area	Local	Production	Where to test
Receipt upload	Direct POST to fake-gcs through Caddy	V4 signed URL, PUT, 5-minute expiry	Staging
Service-to-service auth	Skipped	ID-token check on internal endpoints	Staging
Background jobs	In-process task	Cloud Tasks with OIDC token, dedupe by task name	Staging
Connection limits	Not reached	Pool sizes × max instances vs <code>max_connections</code>	Staging load test

Area	Local	Production	Where to test
Edge	Caddy	Firebase Hosting, later HTTPS LB + Cloud Armor	Staging
User JWT verification	On (emulator tokens)	On (real tokens)	Both
RLS, schema, idempotency	Same SQL	Same SQL	Both

Deployment checklist

- ☐ Create the project and enable APIs: Cloud Run, Cloud SQL Admin, Secret Manager, Cloud Storage, Identity Platform, Artifact Registry.
- ☐ Identity Platform: enable the Google provider; add the Hosting domain to authorized domains.
- ☐ Cloud SQL Postgres: create one instance with two databases, bookkeeping and chat; apply the schema, RLS policies and the `app_user` role with your migration tool.
- ☐ Secret Manager: store the LLM API key and the `app_user` and `chat_user` DB passwords.
- ☐ Create two service accounts: `agent-sa` and `transactions-sa`.
- ☐ Deploy the transactions service with `--allow-unauthenticated` (required by Hosting rewrites; auth is enforced in code), `--service-account transactions-sa`, the Cloud SQL connection, and secrets as env vars.
- ☐ Deploy the agent service with `--allow-unauthenticated`, `--service-account agent-sa`, request timeout raised (for example 300 s), and concurrency around 80 (I/O-bound).
- ☐ Verify every route rejects requests without a valid Firebase JWT, and internal routes without the `agent-sa` ID token.
- ☐ Grant `agent-sa` object read/write on the receipts bucket and `roles/iam.serviceAccountTokenCreator` on itself for signed URLs.
- ☐ Receipts bucket: uniform access, no public access, CORS allowing PUT from the Hosting origin, a lifecycle rule if you don't keep images forever.
- ☐ Firebase Hosting: deploy the SPA with rewrites for `/api/chat/**` and `/api/transactions/**` to the two services; confirm SSE is not buffered end to end.
- ☐ Per-user quotas enabled in the agent service.

- ☐ Before public launch: external HTTPS LB with serverless NEG's and Cloud Armor in front of both services, raised backend timeout on the agent route, and Cloud Run ingress set to `internal-and-cloud-load-balancing` so the services are reachable only through the load balancer.
- ☐ Firestore: create the database in Native mode, deploy `firestore.rules`, and grant `agent-sa` and `transactions-sa` roles/`datastore.user`.
- ☐ Cloud SQL: set the `max_connections` flag to fit the connection budget; set `--max-instances` on both services and pool sizes to match; enable managed connection pooling if on Enterprise Plus.
- ☐ Agent service: `--min-instances=1`.
- ☐ Cloud Tasks: create the `summarize` queue with dispatch rate and concurrency limits; tasks carry an OIDC token for `agent-sa`, verified in code.
- ☐ Langfuse: create staging and prod projects in the EU region, store keys in Secret Manager, set retention and the masking function.
- ☐ Cloud Monitoring alerts (5xx, latency, DB connections, queue age, LLM fallback rate), GCP budget alerts and LLM provider spend caps.
- ☐ Daily scheduled purge of `idempotency_keys` older than 24 hours.
- ☐ At launch: configure Direct VPC egress (all traffic) on the agent service with Cloud NAT, then set ingress to `internal-and-cloud-load-balancing` on both services and run the ingress smoke test in staging first.